Implementation of a Generalized Gas-Phase Combustion Model in Uintah's ICE Component

James Karr

July 9, 2026

1 Introduction

The following model (`gasCombustion`) is a generalization of the hydrogen-only `hydrogenBurke` model to an arbitrary gas-phase reaction mechanism. The species set, thermodynamic data, reaction kinetics, and transport property fits are no longer hard-coded; they are read at problem setup from a mechanism file (Section 2). The document describes the steps and equations used to simulate gas-phase combustion in Uintah's ICE component. The following assumptions and simplifications have been made:

1. The mechanism's N species are tracked through the use of passive scalars. Each passive scalar takes on a value $0 < \phi < 1$ which represents the mass fraction of that species. Only $N - 1$ passive scalars are transported, since one *closure species* (chosen in the ups input file; normally the inert/bath species such as N_2 or Ar, or else a dominant product or reactant) is computed from mass-fraction closure:

$$Y_{\text{closure}} = 1 - \sum_{k \neq \text{closure}} Y_k$$

2. Thermodynamic data (h, g, c_p) is calculated using the NASA 7 polynomials, whose raw coefficients are read per species from the mechanism file.

3. Transport data $(\mu, \lambda, \mathcal{D}_{ij})$ is calculated from polynomials fitted in log space. The fits are generated by the mechanism conversion tooling (Section 2) from the kinetic-theory parameters in the mechanism file, reproducing Cantera's mixture-averaged transport class (`GasTransport`) to within a fraction of a percent.

4. The model modifies the source terms of the governing PDEs (energy and passive scalar sources) and the transport properties (specific heat, specific heat ratio, viscosity, thermal conductivity, and molecular diffusion).

5. In addition, the model *owns the caloric equation of state* of its material: ICE's conserved energy for this material carries the true mixture sensible internal energy $e_s(T, Y)$ rather than $c_v T$, and every temperature $\leftrightarrow$ energy conversion is delegated to the model (Section 11).

1.1 Code layout

The implementation is split into two classes:

- `ReactionMech` (`gasCombustionMechanism.h/.cc`) — a standalone container and evaluator for the mechanism. It parses the mechanism file once and then provides read-only, thread-safe evaluators for thermodynamics, kinetics, and transport. It has no dependence on the grid, DataWarehouse, or scheduler, so it can be unit-tested against Cantera with a small standalone driver.
- `gasCombustion` (`gasCombustion.h/.cc`) — the Uintah coupling: task scheduling, DataWarehouse traffic, the chemistry ODE sub-integrator, species diffusion fluxes, and the caloric-EOS hooks.

2 Mechanism File

The mechanism is supplied as an XML file produced from a standard Cantera YAML mechanism by the conversion script `yaml2xml.py`. The converter copies the species list, elemental compositions, raw NASA 7 coefficients (with their declared temperature ranges), and reaction data (type, reactants/products, Arrhenius parameters, third-body efficiencies, Troe falloff parameters) into a CTML-like XML layout. In addition, `transport_fit.py` recomputes Cantera's 5-coefficient transport polynomial fits from the Lennard-Jones parameters in the YAML (Chapman–Enskog theory with the Monchick–Mason collision integrals) and the converter embeds them in the XML: per-species `<viscosityFit>` and `<conductivityFit>` elements and a `<binaryDiffusionFits>` block with one entry per species pair.

The ups input file selects the mechanism and the closure species:

```
<Model type="gasCombustion">
  <gasCombustion>
    <material>       reactant           </material>
    <mechanismFile>  Burke2012Modified.xml </mechanismFile>
    <closureSpecies> N2                 </closureSpecies>
    <Y_init>         [...]              </Y_init>
    ...
  </gasCombustion>
</Model>
```

Tracked-species ordering (used by `<Y_init>`, `geom_object <Y>` vectors, and initialization profile columns) is the mechanism's species order with the closure species removed.

2.1 Parsing and validation

`ReactionMech::parse` loads the XML with Uintah's `ProblemSpecReader` and runs a sequence of parser stages in dependency order: species indexing, molecular weights (from the elemental composition, not the species name), specific gas constants, pairwise molecular weight tables, binary diffusion fits, reaction lists, chaperon efficiencies, Arrhenius parameters, Troe parameters, NASA 7 polynomials, and viscosity/conductivity fits. A final `validate()` pass checks that every table is sized consistently before the object is used. Bulletproofing enforced at parse time includes:

- the phase must declare `thermo model="ideal-gas"`, `kinetics model="gas"`, and `transport model="mixture-averaged"`;
- the unit system must be cm–mol (Arrhenius A in cm-mol-s units); activation energies are converted to J/mol from the units attribute on each entry (cal/mol, kcal/mol, J/mol, kJ/mol, or K);

- every species listed in the phase must have a data block, a 7-coefficient NASA 7 polynomial per temperature range with a common low/high switch temperature T_{mid}, and 5-coefficient transport fits;
- every binary diffusion pair must be present (the mixture rule divides by $\mathcal{D}_{jk}$);
- only reversible reactions of the supported classes (Section 4) are accepted; irreversible, SRI/Lindemann falloff, PLOG, and non-integer stoichiometry throw with the reaction id.

The mechanism's own declared NASA 7 temperature range is also recorded: T_{low} and T_{high} are the tightest bounds within which *every* species' polynomial is a fit rather than an extrapolation. These bounds replace the previously hard-coded temperature limits in the runtime bulletproofing (Section 8) and in the Newton temperature recovery (Section 11).

3 Reactions

The generalized rate evaluator supports three reaction classes, dispatched per reaction from the mechanism file:

1. **elementary** — arbitrary integer stoichiometry, e.g. $R_1 + R_2 \leftrightarrows P_1 + P_2$ (including cases with a spectator species on both sides, such as $H_2O + H_2O \leftrightarrows H + OH + H_2O$);
2. **three-body** — reactions written with $+M$, e.g. $R_1 + R_2 + M \leftrightarrows P_1 + M$, with per-species chaperon efficiencies;
3. **Troe falloff** — pressure-dependent reactions written with $(+M)$, blending low- and high-pressure Arrhenius limits.

Duplicate reactions (the same equation listed with two Arrhenius parameter sets) are stored and evaluated as separate reactions; because the reverse rate of each is derived from the same equilibrium constant, the summed net rate is algebraically identical to lumping the forward rates (Section 4.1).

As a concrete example, the reference mechanism currently exercised by the model is the hydrogen mechanism of M. Burke et al., *Comprehensive H2/O2 Kinetic Model for High-Pressure Combustion* (Int. J. Chem. Kinet. 44, 2012), with species $(H_2, O_2, N_2, H_2O, H, O, OH, HO_2, H_2O_2)$, 23 reactions after omitting those involving Ar, He, CO, and CO_2, and N_2 as the closure species (`Burke2012Modified.xml`).

4 Reaction Rates

All reaction classes are computed by one generic loop (`ReactionMech::globalRates`). Concentrations are in mol/cm^3 and rates of progress in mol/cm^3·s. For every reaction r with reactant index list $\mathcal{R}$ and product index list $\mathcal{P}$ (indices repeated for stoichiometry):

Forward rate. Modified Arrhenius law:

$$k_f = A\,T^n \exp(-E_a/R_u T)$$

Third-body concentration. If the reaction has chaperon efficiencies α_i (three-body and falloff classes):

$$M = \sum_i \alpha_i \, [s_i]$$

otherwise $M = 1$.

Equilibrium constant and reverse rate. The dimensionless equilibrium constant comes from the NASA 7 Gibbs free energy polynomials, $\hat{g}_k = G_k/(R_u T)$:

$$k_p = \exp\Big(\sum_{k\in\mathcal{R}} \hat{g}_k - \sum_{k\in\mathcal{P}} \hat{g}_k\Big) = \exp(-\Delta G^o/R_u T)$$

It is converted to concentration units through the mole change $\Delta n = |\mathcal{P}| - |\mathcal{R}|$:

$$k_c = k_p \left(\frac{10^{-6}\, P_{\text{ref}}}{R_u T}\right)^{\Delta n}, \qquad P_{\text{ref}} = 101325 \text{ Pa}$$

where $P_{\text{ref}}/R_u T$ is the standard-state concentration in mol/m^3 and the 10^{-6} converts to mol/cm^3. For a balanced reaction ($\Delta n = 0$) this reduces to $k_c = k_p$; unbalanced reactions ($\Delta n = \mp 1$, etc.) pick up the RT/P_{ref} or P_{ref}/RT factors needed to keep the units of k_r consistent. The reverse rate is then $k_r = k_f/k_c$. While all reactions are written as reversible, it is the entropy contribution in Gibbs free energy that ultimately drives reactions from reactants to products in the direction of increasing thermodynamic favorability (i.e., toward equilibrium).

Net rate of progress.

$$\omega_r = M\Big(k_f \prod_{k\in\mathcal{R}} [s_k] - k_r \prod_{k\in\mathcal{P}} [s_k]\Big)$$

Repeated indices produce the correct powers automatically (e.g. $[OH]^2$ for $OH + OH \leftrightarrows O + H_2O$).

4.1 Duplicate Reactions

Reactions listed twice with different Arrhenius parameters are evaluated independently, one entry per parameter set. Since both entries share the same species lists and therefore the same k_c, the summed net rate

$$\omega_a + \omega_b = (k_{f,a} + k_{f,b}) \prod_{\mathcal{R}} [s_k] - \frac{k_{f,a} + k_{f,b}}{k_c} \prod_{\mathcal{P}} [s_k]$$

is algebraically identical to lumping the forward rates into a single reaction, so no special reaction class is required.

5 Third Body Reactions

Third body reactions capture the chemistry other species not present in the reaction might be contributing. This is accomplished through the additional term M defined above, with the efficiency coefficients α_i read per reaction from the mechanism file (`default` attribute for unlisted species, explicit `name:value` overrides otherwise — including explicit zeros, e.g. H_2O:0). The M term multiplies both the forward and reverse contributions, and the Δn correction in k_c keeps the units consistent for unbalanced third-body reactions; no per-shape special cases are required.

6 Troe Falloff Reactions (Pressure dependent)

In pressure dependent reactions, an effective forward rate is calculated from an interpolation between a high and low pressure limit. Calculate the low (k_0, from the `k0` Arrhenius set) and high (k_∞, from the `kInf` set) pressure limits using the modified Arrhenius law, and the third body term M as defined above. Find a reduced pressure:

$$P_r = \frac{k_0 M}{k_\infty}$$

And a Troe Center Factor (the fourth parameter T_2 is optional; its term is included only when the mechanism supplies it):

$$F_c = (1-a)\exp(-T/T_3) + a\exp(-T/T_1) \;\; [+ \exp(-T_2/T)]$$

Next calculate the constants:

$$\begin{aligned} C &= -0.4 - 0.67\log_{10}(F_c) \\ N &= 0.75 - 1.27\log_{10}(F_c) \\ d &= 0.14 \end{aligned}$$

Find the Troe Falloff factor (F) using:

$$\log_{10}(F) = \frac{\log_{10}(F_c)}{1 + \left(\frac{\log_{10}(P_r)+C}{N - d(\log_{10}(P_r)+C)}\right)^2}$$

Finally find the effective forward rate:

$$k_{f,\text{eff}} = k_\infty \frac{P_r}{1+P_r} F$$

$k_{f,\text{eff}}$ replaces k_f in the generic rate expression, with M reset to 1 — in falloff form the third body enters only through P_r, never as a multiplier on the net rate.

7 Functions

All kinetics, thermodynamics, and transport evaluation is provided by generic evaluators on `ReactionMech`. Every evaluator is `const` and takes caller-owned output/scratch vectors (a `Workspace` struct), so the per-cell hot loops perform no heap allocation and the shared mechanism object is safe under Uintah's threaded schedulers.

7.1 globalRates(dbl T, vec C, Workspace w, vec& q)

Purpose: Compute the net rate of progress ω_r (mol/cm^3·s) for every reaction.
Steps:

1. $RT = R_u T$; evaluate $\hat{g}_k$ for every species once per call
2. For each reaction: k_f via modified Arrhenius; M from efficiencies (1 if none); Troe blend replacing k_f for falloff reactions
3. $k_c = k_p\big(10^{-6} P_{\text{ref}}/R_u T\big)^{\Delta n}$; $k_r = k_f/k_c$
4. $q_r = M\big(k_f \prod_{\mathcal{R}} C_k - k_r \prod_{\mathcal{P}} C_k\big)$

7.2 heatRelease(vec q, dbl T, Workspace w)

Purpose: Convert rates of progress to volumetric heat release (W/m^3). Under the constant-volume sub-integration the relevant reaction energy is the net *internal* energy, $\bar{u}_k(T) = \bar{h}_k(T) - R_u T$ (J/mol), evaluated from the NASA 7 enthalpy polynomials.
Steps:

1. Evaluate $\bar{u}_k(T)$ for every species once per call
2. $\dot{q} = \sum_r q_r \left(\sum_{k \in \mathcal{R}} \bar{u}_k - \sum_{k \in \mathcal{P}} \bar{u}_k \right) \times 10^6$ — unit conversion from W/cm^3 to W/m^3

Spectator species appearing on both sides (e.g. the third H_2O in $H_2O + H_2O \leftrightarrows H + OH + H_2O$) cancel automatically.

7.3 massSource(vec q, Workspace w, vec& S)

Purpose: Calculate species mass source terms (kg/m^3·s) from the rates of progress.
Steps:

1. Molar production rates: $\dot{s}_k \mathrel{-}= q_r$ per reactant occurrence and $\dot{s}_k \mathrel{+}= q_r$ per product occurrence, over all reactions (repetition encodes the stoichiometric coefficients $\nu_{r,k}$)
2. $S_j = M_{w,k}\, \dot{s}_k \times 10^3$ — convert mol/cm^3·s to kg/m^3·s, returned in tracked-species order (closure species skipped)

7.4 Thermodynamic evaluators

- `cpSpecificHeat(T, cp)` — dimensionless $\hat{c}_{p,k}(T)$ for every species from the NASA 7 polynomials (low/high range switched at T_{mid}).
- `sensibleEnthalpyAllSpecies(T, hs)` — sensible enthalpies $h_{s,k}(T) = h_k(T) - h_k(T_0)$ [J/kg], $T_0 = 298.15$ K.
- `sensibleEnergy(T, Y)` — mixture sensible internal energy $e_s(T, Y) = \sum_k Y_k \big[h_{s,k}(T) - R_k(T - T_0) \big]$ [J/kg].
- `temperatureFromSensibleEnergy(es, Y, Tguess)` — Newton inversion of $e_s(T, Y) = e_s$ (Section 11).

7.5 Transport evaluators

`viscosity(T, X, w)`, `thermalConductivity(T, X)`, and `mixtureAvgDiffCoeffs(T, rho, Y, w, Dk)` implement the mixing rules of Section 9 from the polynomial fits read out of the mechanism file.

7.6 chemStep (model side)

`gasCombustion::chemStep(T, Y, rho, cellVol, res)` assembles the constant-volume chemistry right-hand side at a state (T, Y): mixture c_v from `cpSpecificHeat`, molar concentrations $C_k = 10^{-3} \rho Y_k / M_{w,k}$, then `globalRates`, `massSource`, and `heatRelease`, returning $\dot{Y}_k = S_k/\rho$, $\dot{T} = \dot{q}/(\rho c_v)$, and the energy source rate $\dot{q}\, V_{\text{cell}}$.

8 Implementation

Source terms are computed in `computeModelSources`, which is called once per advection timestep. Rather than computing instantaneous rates and applying them directly, chemistry is integrated cell-by-cell over the full advection interval Δt_{adv} using a constant-volume *adaptive* ODE sub-stepping loop. This decouples the stiff chemistry timescale from the larger fluid timestep imposed by ICE.

8.1 Step 1: Build the Cell State / Bulletproofing

For each cell retrieve from the data warehouse: Δt_{adv}, temperature T, density ρ, and the $N-1$ tracked species mass fractions. Compute the closure fraction and insert it at the closure index to form the full N-species vector:

$$Y_{\text{closure}} = 1 - \sum_{k \neq \text{closure}} Y_k$$

Bulletproofing checks are applied: $\rho > 0$ and $Y_k \geq 0$ for all species. The temperature checks are derived from the mechanism's own declared NASA 7 range rather than hard-coded: a warning is issued outside $[T_{\text{low}},\, T_{\text{high}}]$ (where some species' polynomial becomes an extrapolation), and the run aborts outside the wider sanity band $[0.6\,T_{\text{low}},\, 1.4\,T_{\text{high}}]$. For the Burke 2012 mechanism these evaluate to warn outside $[300, 3500]$ K and abort outside $[180, 4900]$ K.

8.2 Step 2: Constant-Volume Adaptive ODE Integration ($t = 0 \rightarrow \Delta t_{\text{adv}}$)

The integrator is a Heun–Euler embedded pair (RK2(1)) with adaptive step-size control. Each trial sub-step of size Δt_{chem} proceeds as follows (the final sub-step is clamped so the loop ends exactly at Δt_{adv}):

8.2.1 2a: Predictor (1st order)

Evaluate the RHS $f =$ `chemStep` at the start state (once per outer step, reused on every retry), then take a full Euler step:

$$Y_k^* = Y_k^n + \Delta t_{\text{chem}}\, \dot{Y}_k^n, \qquad T^* = T^n + \Delta t_{\text{chem}}\, \dot{T}^n$$

with the closure species updated by difference.

8.2.2 2b: Corrector (2nd order, carried solution)

Evaluate the RHS again at the predictor state and average:

$$Y_k^{n+1} = Y_k^n + \tfrac{1}{2}\Delta t_{\text{chem}}\big(\dot{Y}_k^n + \dot{Y}_k^*\big), \qquad T^{n+1} = T^n + \tfrac{1}{2}\Delta t_{\text{chem}}\big(\dot{T}^n + \dot{T}^*\big)$$

8.2.3 2c: Error control

The difference between corrector and predictor is the embedded local error estimate. A weighted RMS norm is formed over the whole integrated state (all tracked species plus temperature; the closure species is set by difference, not integrated):

$$\text{err} = \sqrt{\frac{1}{N_{\text{norm}}} \sum_i \left(\frac{y_i^{n+1} - y_i^*}{r_{\text{tol}}\, |y_i^{n+1}| + a_{\text{tol}}} \right)^2}$$

with separate absolute-tolerance floors for mass fractions ($a_{\mathrm{tol},Y}$) and temperature ($a_{\mathrm{tol},T}$); the floor stops a trace radical near zero from forcing spurious rejections. The step is accepted if err ≤ 1, and the next step size is

$$\Delta t_{\mathrm{chem}} \leftarrow \Delta t_{\mathrm{chem}} \min\!\big(f_{\mathrm{grow}},\, \max(f_{\mathrm{shrink}},\, s\,\mathrm{err}^{-1/2})\big)$$

(safety factor s, growth/shrink clamps from the ups input; the exponent $-1/2$ reflects the 1st-order error estimate). Rejected steps shrink and retry from the same start state. Per-cell diagnostics are stored: the smallest accepted sub-step (`dt_chemistry`), which variable limited it (`dt_chemistry_limiter`), and the mean heat release rate (`HeatReleaseRate`).

8.3 Step 3: Write Source Terms

After the integration loop completes, the accumulated terms are added to the ICE source arrays:

- $Y_{\mathrm{src},k}$ += massSrc$_k$ (dimensionless mass fraction source)
- `eng_src` += $\rho\, V_{\mathrm{cell}}\big[e_s(T_{\mathrm{end}}, Y_{\mathrm{end}}) - e_s(T_{\mathrm{start}}, Y_{\mathrm{start}})\big]$ (J)

The energy source is the *exact state-function difference* of the sensible energy over the constant-volume sub-integration, not the path integral $\int \dot{q}\,\mathrm{d}t$ — see Section 11. The path integral survives only as the `HeatReleaseRate` diagnostic.

9 Thermo-Transport Properties

Separately from the source term calculation, `modifyThermoTransportProperties` is scheduled to run before each timestep solve. It loops over all cells (including ghost cells) and evaluates $c_{p,\mathrm{mix}}$, R_{mix}, c_v, γ, μ, λ, and the mixture-averaged diffusion coefficients D_k from the current temperature and species mass fractions, writing the results to ICE's `specific_heatLabel` (c_v), `gammaLabel` (γ), `viscosityLabel` (μ), `thermalCondLabel` (λ), and per-species `diffCoef` labels. ICE reads these labels before the solving process, so the mixture thermodynamic properties remain consistent with the evolving composition throughout the simulation.

9.1 Specific Heat and Ratio of Specific Heats

Non-dimensional species constant-pressure specific heats are evaluated from the NASA 7 polynomials. For each species k the polynomial form is:

$$\hat{c}_{p,k} = a_{0,k} + a_{1,k}T + a_{2,k}T^2 + a_{3,k}T^3 + a_{4,k}T^4$$

Mixture averages are then assembled as:

$$c_{p,\mathrm{mix}} = \sum_k Y_k R_k \hat{c}_{p,k}, \qquad R_k = \frac{R_u}{M_{w,k}} \quad [\mathrm{J/kg\text{-}K}]$$
$$R_{\mathrm{mix}} = \sum_k Y_k R_k \quad [\mathrm{J/kg\text{-}K}]$$

Constant-volume specific heat and ratio of specific heats follow from ideal gas relations:

$$c_v = c_{p,\mathrm{mix}} - R_{\mathrm{mix}}, \qquad \gamma = \frac{c_{p,\mathrm{mix}}}{c_v}$$

9.2 Viscosity

Mixture viscosity is calculated using the Wilke mixing rule. Pure species viscosities η_k are first evaluated from the polynomial fits in $\ln T$ read from the mechanism file:

$$\sqrt{\eta_k} = T^{1/4}\big(\mu_{0,k} + \mu_{1,k}\ln T + \mu_{2,k}\ln^2 T + \mu_{3,k}\ln^3 T + \mu_{4,k}\ln^4 T\big)$$

Interaction coefficients ϕ_{ij} between species i and j are then formed:

$$\phi_{ij} = \frac{\left(1 + \sqrt{\eta_i/\eta_j}\,(M_{w,j}/M_{w,i})^{1/4}\right)^2}{\sqrt{8\,(1 + M_{w,i}/M_{w,j})}}$$

(the pairwise molecular-weight factors are precomputed once at parse time). Finally the mixture viscosity is assembled via the Wilke mixing rule:

$$\mu_{\mathrm{mix}} = \sum_i \frac{X_i\,\eta_i}{\sum_j \phi_{ij}\,X_j}$$

9.3 Thermal Conductivity

Mixture thermal conductivity is evaluated using the arithmetic-harmonic mean of the pure species conductivities. Pure species conductivities λ_k are evaluated from the polynomial fits in $\ln T$:

$$\lambda_k = T^{1/2}\big(k_{0,k} + k_{1,k}\ln T + k_{2,k}\ln^2 T + k_{3,k}\ln^3 T + k_{4,k}\ln^4 T\big)$$

Arithmetic and harmonic mixture averages are computed over all species mole fractions:

$$\lambda_{\mathrm{arith}} = \sum_k X_k\,\lambda_k, \qquad \lambda_{\mathrm{harm}} = \left(\sum_k \frac{X_k}{\lambda_k}\right)^{-1}$$

The mixture thermal conductivity is taken as the mean of these two estimates:

$$\lambda_{\mathrm{mix}} = \frac{1}{2}\left(\lambda_{\mathrm{arith}} + \lambda_{\mathrm{harm}}\right)$$

9.4 Mixture-Averaged Diffusion Coefficients

Mixture-averaged diffusion coefficients are computed following Kee et al. *Chemically Reacting Flow* Eq. 12.176, which is the algorithm used in Cantera's `getMixDiffCoeffs`. The procedure begins with binary diffusion coefficients evaluated from degree-4 polynomial fits in $\ln T$:

$$\mathcal{D}_{ij}(T,P) = \frac{T^{3/2}}{P}\big(d_{0,ij} + d_{1,ij}\ln T + d_{2,ij}\ln^2 T + d_{3,ij}\ln^3 T + d_{4,ij}\ln^4 T\big)$$

where the polynomial coefficients $d_{n,ij}$ are the unit-pressure fits embedded in the mechanism file and the factor of $1/P$ scales to the actual pressure. The mixture mean molar mass is:

$$\bar{M} = \sum_k X_k M_{w,k}$$

The mixture-averaged diffusion coefficient for species k is then:

$$D_k = \frac{\bar{M} - X_k M_{w,k}}{P\,\bar{M}\displaystyle\sum_{j\neq k} X_j/\mathcal{D}_{jk}}$$

This expression weights each binary pair by the mole fraction of the partner species and accounts for the fact that species k diffuses through the mixture of all other species.

10 Molecular Diffusion with Correction Velocity

When diffusion is enabled, `computeModelSources` adds the divergence of the species diffusion fluxes to the passive scalar sources and the corresponding sensible-enthalpy flux to the energy source. All fluxes are evaluated at cell faces.

10.1 Uncorrected fluxes

Cell-centered mole fractions are assembled from the tracked scalars (closure species by difference),

$$X_k = \frac{Y_k}{M_{w,k} \sum_j Y_j / M_{w,j}} = \frac{\bar{M}}{M_{w,k}} Y_k,$$

and their face-normal gradients ∇X_k are computed with ICE's flux operator using unit coefficients (so no diffusivity is baked into the gradient). With face-averaged ρ, T, Y, and $\bar{M}$, and the mixture-averaged coefficients D_k of Section 9 evaluated at the face state, the uncorrected (Hirschfelder–Curtiss) mass flux of species k is

$$\mathbf{j}_k^* = -\rho\, D_k \frac{M_{w,k}}{\bar{M}} \nabla X_k \qquad [\mathrm{kg/m^2 s}]$$

10.2 Correction velocity

The mixture-averaged approximation does not guarantee that the fluxes sum to zero: in general $\sum_k \mathbf{j}_k^* \neq 0$, which would create or destroy mass. Mass conservation is restored with a correction velocity $\mathbf{V}_c$ common to all species,

$$\mathbf{V}_c = -\frac{1}{\rho} \sum_j \mathbf{j}_j^*, \qquad \mathbf{j}_k = \mathbf{j}_k^* + \rho\, Y_k\, \mathbf{V}_c = \mathbf{j}_k^* - Y_k \sum_j \mathbf{j}_j^*,$$

so that $\sum_k \mathbf{j}_k = 0$ identically. The implementation verifies $|\sum_k \mathbf{j}_k| < 10^{-12}$ at every face and aborts otherwise. Note that the correction is applied over *all* N species including the closure species — the closure species' flux is computed like any other and participates in the correction, so closure-by-difference remains consistent with the diffused fields.

10.3 Source terms

The corrected fluxes enter the tracked-species scalar sources through their divergence (face-area weighted, per unit mass):

$$Y_{\mathrm{src},k} \mathrel{-}= \frac{\nabla\cdot \mathbf{j}_k\, \Delta t_{\mathrm{adv}}}{\rho}$$

Species carrying different sensible enthalpies also transport energy. The face sensible-enthalpy flux is

$$\boldsymbol{\phi} = \sum_k \bar{h}_{s,k}\, \mathbf{j}_k \qquad [\mathrm{W/m^2}]$$

with $\bar{h}_{s,k}$ the arithmetic face average of the NASA 7 sensible enthalpies of the two adjacent cells, and its divergence is deposited into the energy source:

$$\texttt{eng\_src} \mathrel{-}= \nabla\cdot\boldsymbol{\phi}\, V_{\mathrm{cell}}\, \Delta t_{\mathrm{adv}} \qquad [\mathrm{J}]$$

Because $\sum_k \mathbf{j}_k = 0$, the enthalpy flux is a pure interdiffusion term: it vanishes where composition is uniform and transports no net mass anywhere.

11 Energy Equation and Temperature Recovery

This model requires an energy formulation different from ICE's native one. The derivation, quantitative justification, and validation are documented in *Cause 1: Caloric Inconsistency of ICE's Energy Carrier* (`cause1_caloricInconsistency.tex`); this section summarizes the formulation as it applies to the generalized model.

11.1 Why ICE's carrier is insufficient

ICE constructs its conserved Lagrangian energy from the cell-centered specific heat and temperature, $E_{\mathrm{ICE}} = m\, c_v(T,Y)\, T$, and recovers temperature after advection by the inverse operation $T = E/(m\, c_v)$. For the constant-c_v materials ICE was designed around, $c_v T$ *is* the sensible internal energy and the scheme is exact. This model, however, supplies a per-cell, per-timestep mixture $c_v(T,Y)$ from the NASA 7 polynomials. The true specific sensible internal energy is the caloric state function

$$e_s(T,Y) = \sum_k Y_k \int_{T_0}^{T} c_{v,k}(T')\,\mathrm{d}T' = \sum_k Y_k \big[\, h_{s,k}(T) - R_k(T - T_0)\big], \qquad T_0 = 298.15\ \mathrm{K},$$

and because $c_{v,k}$ depends on T, the difference $\delta(T,Y) = c_v(T,Y)\,T - e_s(T,Y)$ is not constant — it has a large gradient across a flame where both T and Y change. Advecting $c_v T$ therefore introduces a spurious advective energy sink concentrated at the flame front; for the 1D stoichiometric hydrogen flame this sink amounted to $\sim$17% of the chemistry heat release and a $\sim$200–340 K underprediction of the burned temperature.

11.2 Caloric EOS ownership

The correction makes the model own the caloric equation of state of its material. ICE's own conserved energy carries $m\, e_s(T,Y)$ for any material whose model claims ownership through the `FluidsBasedModel` interface:

- `ownsCaloricEOS(indx)` — true for the model's material;
- `computeSensibleEnergy(es, temp, ...)` — evaluates $e_s(T,Y)$ per cell;
- `computeTempFromSensibleEnergy(temp, es, ...)` — inverts it.

Both hooks receive a `YForm` argument identifying which DataWarehouse form the composition scalars are in at the call site (old-timestep primitives, mass-weighted Lagrangian scalars, or new-timestep primitives), guaranteeing e_s is always evaluated with the Y simultaneous with the energy being converted. Exactly three sites on the ICE solve path convert between temperature and energy, and only those three fork for an owned material:

1. **Lagrangian build** (`ICE::computeLagrangianValues`): $E_L = m\, e_s(T^n, Y^n)$+`int_eng_source`+ `modelEng_src`.
2. **Multi-material exchange** (`MultiMatlExchange/Scalar.cc`): unpack T from $e_s = E_L/m_L$, perform the heat-exchange solve in temperature space, repack $E_L = m_L\, e_s(T^{ME}, Y_L)$.
3. **Temperature backout** (`ICE::conservedtoPrimitive_Vars`): Newton inversion of $e_s(T^{n+1}, Y^{n+1}) = E_{\mathrm{adv}}/m_{\mathrm{adv}}$.

The physical c_v and γ fields are unchanged, so the thermal EOS $P = \rho RT$, the sound speed, compressibility, LODI boundary conditions, and time-step control all remain exactly as before.

11.3 Chemistry source as an exact state difference

Over the operator-split, constant-volume chemistry sub-integration the total internal energy of the cell is constant, so the exact integral of the chemistry source is the state-function difference

$$\Delta e_s\big|_{\text{chem}} = e_s\big(T_{\text{end}}, Y_{\text{end}}\big) - e_s\big(T_{\text{start}}, Y_{\text{start}}\big) = -\sum_k u_k(T_0)\,\Delta Y_k.$$

This is what `computeModelSources` deposits into `modelEng_src` (Section 8, Step 3). Because the source is itself an e_s difference, the Lagrangian build telescopes to $m\,e_s(T^n, Y^n) + \texttt{modelEng\_src} = m\,e_s(T_{\text{chem}}, Y_{\text{chem}})$ with no path-integral (quadrature) error, and it automatically contains the composition-shift contribution $\sum_k e_{s,k}\,\mathrm{d}Y_k$ that the path integral $\int \dot{q}\,\mathrm{d}t = \int \rho\, c_v\,\mathrm{d}T$ omits.

11.4 Temperature recovery

With Y^{n+1} and e_s^{n+1} in hand, temperature is recovered by Newton iteration on the caloric EOS:

$$T \leftarrow T - \frac{e_s(T, Y^{n+1}) - e_s^{n+1}}{c_v(T, Y^{n+1})}, \qquad c_v(T, Y) = \sum_k Y_k R_k\big(\hat{c}_{p,k}(T) - 1\big)$$

The iteration is globally convergent from any in-range guess since $\partial e_s/\partial T = c_v > 0$ makes e_s monotone in T. The initial guess is clamped to the mechanism's declared NASA 7 range $[T_{\text{low}}, T_{\text{high}}]$ and the iterates to the wider sanity band around it (Section 8); convergence to a relative tolerance of 10^{-10} in T typically takes 3–5 iterations, and failure to converge in 50 iterations throws rather than clamping — a genuinely unphysical state should stop the run, not be hidden. Note that $e_s < 0$ is a legal state (temperatures below T_0), so ICE's negative-energy safety nets are bypassed for owned materials.